

IoT Smart Bell & Announcement System Advanced Documentation

1. Introduction

This project is a full-stack IoT automation system designed to manage institutional bell ringing and audio announcements. It integrates a Node.js backend, MQTT communication, and ESP32-based audio nodes to deliver reliable and scalable operation across a local network.

2. Problem Statement

Traditional systems rely on manual intervention and lack flexibility. There is no centralized control, scheduling is inconsistent, and announcements are limited. A smart, automated, and network-based solution is required.

3. Objectives

- 1 Automate bell ringing using schedules
- 2 Enable real-time and scheduled announcements
- 3 Ensure lightweight execution on ESP32
- 4 Use reliable communication (MQTT)
- 5 Provide scalable architecture for multiple nodes

4. System Architecture

The system consists of three layers:

- 1 Backend Layer: Node.js server for logic, scheduling, and APIs
- 2 Communication Layer: MQTT broker for message passing
- 3 Hardware Layer: ESP32 + MAX98357A amplifier + speaker

5. Detailed Workflow

1. User triggers API or schedule 2. Backend processes request 3. Audio prepared or selected 4. MQTT publishes PLAY_URL message 5. ESP32 receives message 6. ESP32 downloads audio via HTTP 7. WAV parsed and PCM extracted 8. Audio streamed via I2S 9. Amplifier outputs sound

6. Backend Design (Deep Explanation)

The backend is built using Node.js and Express. It follows a modular architecture including routes, controllers, and services.

- 1 Routes: Handle API endpoints
- 2 Controllers: Process request logic
- 3 Services: Handle audio processing, MQTT, scheduling
- 4 Static Media Server: Hosts audio files

- 5 Cron Jobs: Execute scheduled announcements

7. MQTT Communication (Deep)

MQTT uses a publish-subscribe model.

- 1 Backend acts as publisher
- 2 ESP32 acts as subscriber
- 3 Topic example: school/audio
- 4 Message format: PLAY_URL|http://server/media/file.wav
- 5 Ensures low-latency and lightweight communication

8. Audio Processing Pipeline (Detailed)

Audio is standardized before playback.

- 1 Input: MP3 or other formats
- 2 Converted using FFmpeg
- 3 Output: WAV (16kHz, mono, PCM s16le)
- 4 Reason: Simple decoding on ESP32
- 5 Ensures stable playback without distortion

9. ESP32 Audio Handling

ESP32 downloads audio file and processes it manually.

- 1 HTTP GET request to fetch audio
- 2 Parse WAV header
- 3 Extract PCM data
- 4 Stream data to I2S driver
- 5 Continuous playback via DMA buffers

10. Hardware Explanation

- 1 ESP32: Microcontroller with WiFi and I2S support
- 2 MAX98357A: Digital-to-analog audio amplifier
- 3 Speaker: Outputs sound
- 4 I2S Pins: DOUT=25, BCLK=26, LRC=27

11. Memory Optimization Strategy

ESP32 has limited RAM, especially without PSRAM.

- 1 Avoid heavy libraries like Audio.h
- 2 Manual WAV decoding
- 3 Streaming instead of full buffering
- 4 Minimal memory footprint

12. Scheduling System

Schedules are stored and converted into cron expressions.

- 1 Dynamic schedule creation
- 2 Enable/disable schedules
- 3 Automatic execution via cron jobs
- 4 Supports daily/weekly patterns

13. Failure Handling

- 1 Network failure → retry mechanism
- 2 Invalid audio → fallback handling
- 3 MQTT disconnect → auto reconnect
- 4 File missing → error logging

14. Advantages

- 1 Fully automated
- 2 Scalable architecture
- 3 Low resource usage
- 4 Real-time control
- 5 Easy deployment

15. Limitations

- 1 Dependent on network stability
- 2 Limited audio format support
- 3 ESP32 memory constraints

16. Future Enhancements

- 1 Mobile/Web dashboard
- 2 Cloud MQTT integration
- 3 Multi-zone audio control
- 4 Offline caching
- 5 Improved audio quality

17. Conclusion

This project demonstrates a real-world IoT system integrating embedded systems, backend development, and networking. It is optimized for performance, scalability, and reliability under hardware constraints.